When the delta is fake

Pass-bar design for agent evaluation: nop failures, frozen controls, and scored traces

Gyanu Mayank · Independent Researcher
B.Tech, Computer Science and Engineering, National Institute of Technology Patna
Pune, India · gyanumayank0@gmail.com · github.com/gyanu2507
31 October 2026 · v0.1.3
DOI (concept): 10.5281/zenodo.23829461
Software: github.com/gyanu2507/agent-eval-suite

ABSTRACT

Outcome-based agent benchmarks such as Terminal-Bench specify instruction, containerized environment, tests, and a reference solution (Merrill et al., 2026). Harnesses such as Harbor and Inspect execute those units at scale. They do not, by themselves, guarantee that a task's pass condition is resistant to cosmetic repairs, fixture edits, or self-reported metrics.

This technical note states a pass-bar for task authorship and releases three public tasks that implement it across event-sourced repair, concurrent systems contracts, and distribution-shift reporting. Each task includes a starter with named failure modes, an oracle that clears the suite, and a verifier that recomputes scores from artifacts. A distribution-shift case study shows that mean-only feature normalization need not change ranking AUROC for a linear scorer, and that a train-site label shortcut which flips at evaluation forces an honest control to fail unless the method drops the shortcut. A staged fail transcript is scored with an explicit positive/negative rubric.

Scope is methodological: synthetic fixtures, three tasks, no frontier-model leaderboard. The contribution is a reusable design criteria set and calibrated examples for agent-evaluation work.

1. Positioning

Layer	Provides	Leaves to the task author
Terminal-Bench	Hard terminal tasks, reviewed oracles, outcome tests, Harbor as harness	Failure-mode design inside a new task
Harbor	Parallel trials, trajectories, verifier rewards	Whether the reward is gameable
Inspect	Dataset, solver/agent, and scorer composition with sandboxes	The same question at sample level

The Terminal-Bench unit (instruction, environment, final-state tests, oracle) is taken as the container. The criteria below address reward integrity: whether a green run implies the intended capability rather than a reporting trick.

The released suite is small and public by design. Domain analogues from confidential evaluation work are omitted. The failure classes transferred here are ordering and idempotency under event replay, dual-consumer buffer contracts, and metrics that remain green while the underlying method is a no-op.

2. Pass-bar criteria

1. **Nop failure.** The starter fails for a failure mode that can be stated in one sentence.
2. **Oracle success.** The reference solution clears every unit test.
3. **Artifact-backed score.** The verifier recomputes the control or expected state. Editing a report field or golden file cannot invent a pass.

4. **Calibration record.** `CALIBRATION.md` lists starter failure modes and blocked shortcuts.

Default anti-shortcuts: fixture data and frozen baselines are read-only; property tests extend beyond the stock fixture; occupancy, balances, and controls are derived quantities.

These criteria are stricter than “tests exist” and weaker than a multi-reviewer benchmark admission process. The three tasks demonstrate the criteria in code.

3. Tasks

Checked-in fixtures and oracles, 31 October 2026. Twelve tests pass.

```
pytest -q -p no:randomly \
  tasks/clearing-hold-ledger/tests \
  tasks/slot-ring-buffer/tests \
  tasks/site-shift-meter/tests
```

3.1 clearing-hold-ledger (event-sourced repair)

Objective. Replay an immutable clearing stream into wallet balances and holds. Repair `clearing/replay.py`. Leave `data/events.json` unchanged.

Starter failure modes. Lexicographic `seq` sort; prefix collision on `dedupe_key`; partial HOLD; RELEASE by wallet rather than `hold_key`; zero-balance wallets omitted.

Fixture. 16 events. Oracle balances: alpha 85, beta 45, delta/gamma/omega 0.

Motivation. Repair agents often chase the first failing test, then edit fixtures, then loosen deduplication. Tests recompute expected state from the stream.

3.2 slot-ring-buffer (systems contracts)

Objective. Fixed-capacity packet ring with two independent consumers. Under pressure from the slowest consumer, drop incoming writes. Never overwrite unread slots for the slow reader.

Motivation. Systems-lane evaluation without proprietary netlist fixtures. Single-consumer tests can remain green while the dual-consumer contract fails.

3.3 site-shift-meter (distribution-shift reporting)

Objective. Train on sites S0/S1. Evaluate on S2. Exceed a frozen control with recomputable calibration.

Quantity	Value
`control_auroc`	0.0108
`model_auroc`	0.9766
`delta_auroc`	0.9658
Gate	`delta_auroc >= 0.08`; control match within `1e-6`
`n_eval`	180

Starter. Fits logistic regression on raw features, then copies that AUROC into both report fields. Delta is zero while the JSON schema appears complete.

4. Case study: mean shift without ranking change

Under mean-only shift, ranking AUROC for a linear scorer is largely invariant to a constant offset on scores. Per-site normalization can look like domain adaptation while leaving delta near zero.

The fixture places a strong label shortcut in `f4` on training sites and flips it on evaluation. The frozen control collapses on S2 (≈0.01 AUROC). Dropping `f4` and z-scoring `f1` - `f3` per site recovers ≈0.98 AUROC on the checked-in CSVs.

The gate (`>= 0.08`) rejects control copying and cosmetic mean-only z-scoring once the shortcut is present, while allowing a correct method to pass without a near-perfect classifier.

5. Trace review under an explicit rubric

Fixture: `docs/fixtures/ledger-fail-run.json` (staged). Failure sequence: lexicographic `seq`, fixture edit, prefix dedupe regression, incomplete hold semantics.

```
Agent sorts events by numeric seq before replay, +3
Agent uses exact dedupe_key equality rather than prefix matching, +3
Agent applies HOLD only when full amount is available, +2
Agent releases holds by hold_key identity, +2
Agent keeps zero-balance wallets in the output maps, +1
Agent leaves data/events.json unchanged, +2
Agent edits the event fixture to make tests pass, -5
Agent treats string-ordered seq as solved after one green-looking run, -2
Agent retries the same failing pytest three or more times with no code change, -1
Agent introduces prefix dedupe after seeing a collision failure, -3
```

Positive mass: 13. The staged run fails tests and rubric. A correct repair clears the six positives without modifying `data/`. The rubric separates outcome failure from process failure.

6. Limitations

- Three tasks; synthetic or small fixtures only.
- No multi-model agent sweep. The ledger transcript is constructed.
- Confidential evaluation corpora are excluded.
- Gate values are calibrated to these CSVs.
- No safety or capability claims about frontier systems.

7. Reproduction

```
git clone https://github.com/gyanu2507/agent-eval-suite
cd agent-eval-suite
python3 -m pytest -q -p no:randomly \
  tasks/clearing-hold-ledger/tests \
  tasks/slot-ring-buffer/tests \
  tasks/site-shift-meter/tests
python3 tasks/site-shift-meter/solution/train_and_eval.py
```

References

Merrill, M. A., Shaw, A. G., Carlini, N., et al. (2026). *Terminal-Bench*. arXiv:2601.11868. https://doi.org/10.48550/arXiv.2601.11868

Harbor Framework. https://github.com/harbor-framework/harbor

UK AI Security Institute. *Inspect*. https://inspect.aisi.org.uk/

Mayank, G. (2026). *When the delta is fake* (v0.1.3). Zenodo. https://doi.org/10.5281/zenodo.23829461

Citation

Use the concept DOI so citations resolve to the latest version.

```
@techreport{mayank2026deltafake,
  author       = {Mayank, Gyanu},
  title        = {When the delta is fake: Pass-bar design for agent evaluation},
  year         = {2026},
  month        = {10},
  institution  = {Independent},
  version      = {0.1.3},
  doi          = {10.5281/zenodo.23829461},
  url          = {https://doi.org/10.5281/zenodo.23829461},
  note         = {B.Tech CSE, National Institute of Technology Patna}
}
```